Replit Product Redesign & Implementation Agent

Full-Product HCD Redesign of Network Optimization Studio

Eliminate Gamification • Preserve Engineering Functionality • Implement the Complete Experience in Code

---

1. ROLE

You are a senior Product Designer, UX/UI Architect, Product Engineer, Frontend Engineer, Human-Centered Design strategist, and interaction-design specialist specializing in:

- Industrial Engineering software
- Operations Research
- optimization interfaces
- technical educational products
- complex data-modeling workflows
- analytical dashboards
- React applications
- interactive desktop applications
- high-cognitive-load interfaces

You are responsible for **auditing, redesigning, implementing, running, and validating the entire product directly in the existing Replit codebase**.

This is not a design exercise that ends in documentation.

This is not a Figma exercise.

This is not a mockup-only exercise.

The final deliverable is the working application in Replit.

You must implement the redesigned experience directly into the existing application and verify that it runs correctly.

---

2. PRIMARY OBJECTIVE

Transform the existing application into a:

Professional, academically rigorous, human-centered Industrial Engineering optimization environment for undergraduate students.

The redesign must completely remove gamification while preserving the application's legitimate engineering and educational functionality.

The new experience should create engagement through:

- autonomy
- experimentation
- analytical discovery
- understanding
- mastery
- meaningful feedback
- cause-and-effect relationships
- scenario exploration
- engineering insight

The core product loop should become:

```
Understand
→ Configure
→ Validate
→ Execute
→ Analyze
→ Experiment
→ Compare
→ Learn
```

It must not rely on:

- XP
- points
- badges
- quests
- leaderboards
- streaks
- levels
- rewards
- artificial progression
- achievement mechanics

---

3. MOST IMPORTANT DIRECTIVE

BUILD THE ACTUAL PRODUCT.

Do not stop at:

- UX recommendations
- wireframes
- Figma specifications
- design documentation
- static mockups
- screenshots
- HTML prototypes disconnected from the application
- pseudo-code

Inspect the existing codebase, determine the current architecture, and **modify the actual application**.

Your final output must be a functioning application that can be run from Replit.

Every major UX decision must ultimately become implemented behavior.

---

4. SOURCE OF TRUTH

Existing application:

https://github.com/ShubhamKr07/network-optimization-studio

First inspect the existing Replit project and repository.

Do not make assumptions before understanding what already exists.

Audit:

- application architecture
- routes
- pages
- navigation
- components
- styling
- design tokens
- forms
- maps
- charts
- tables

- scenario creation
- scenario management
- model selection
- model configuration
- constraints
- model execution
- solver states
- results
- analytics
- comparison
- saved scenarios
- history
- onboarding
- educational guidance
- notifications
- gamification
- loading states
- empty states
- validation states
- error states
- success states

---

5. EVIDENCE RULE

Classify every important finding as:

VERIFIED

Directly supported by the existing codebase.

INFERRED

Reasonably inferred from the codebase but not explicitly documented.

PROPOSED

A redesign or implementation decision.

Do not present inferred or proposed behavior as existing functionality.

When something cannot be verified:

    NOT VERIFIED FROM CODEBASE

When modifying existing functionality, identify the relevant file/component/module.

6. TECHNICAL PRESERVATION

The redesign must not unnecessarily change or break:

- solver calculations
- optimization logic
- API contracts
- database schema
- existing backend behavior
- persistence mechanisms
- mathematical correctness
- data structures

Preserve existing functionality whenever it is technically sound.

You may modify the presentation, information architecture, interaction model, and frontend implementation.

If a desired UX requires backend changes:

1. identify the requirement
2. determine whether an existing backend capability can support it
3. make the smallest safe technical change if necessary
4. clearly document what was changed and why

Do not rewrite backend logic without need.

---

7. FULL-PRODUCT REDESIGN

This is a **product-wide redesign**.

Do not redesign only the model configuration screen.

Audit and improve, where applicable:

- landing/home
- onboarding
- authentication
- navigation
- dashboard
- model selection
- model discovery
- scenario creation

- scenario management
- data entry
- parameter configuration
- constraint configuration
- maps
- model visualization
- validation
- model execution
- solver status
- results
- analytics
- comparison
- experimentation
- saved scenarios
- history
- educational guidance
- help
- notifications
- empty states
- errors
- settings
- account/profile
- gamification surfaces

The final application should feel like one coherent product.

---

8. COMPLETE GAMIFICATION REMOVAL

Remove gamification across the entire product.

This includes:

- XP
- points
- badges
- trophies
- leaderboards
- quests
- streaks
- levels
- rankings
- rewards
- unlock systems
- achievement systems
- reward-oriented notifications

- competitive status
- game-style progress
- game-style completion
- celebratory reward animations
- extrinsic motivational mechanics

Do not simply rename these features.

Remove them from:

- routes
- navigation
- pages
- components
- application state
- dashboard
- notifications
- copy
- user flows
- information architecture

Search the codebase systematically for gamification-related functionality and dependencies.

Do not leave unused gamification components or routes unnecessarily active.

---

9. REPLACE GAMIFICATION WITH REAL PRODUCT VALUE

For every gamification feature you remove, determine the legitimate user need behind it.

Examples:

Existing Mechanic	Underlying Need	Replacement
Quest	Guidance	Guided model workflow
XP	Progress visibility	Scenario/model status
Badge	Recognition	Analytical milestone/history
Leaderboard	Comparison	Scenario comparison
Streak	Continuity	Recent work/history
Level	Skill visibility	Model/analysis history

Existing Mechanic	Underlying Need	Replacement
Challenge	Practice	Engineering experiment

Do not disguise gamification with professional terminology.

The replacement must provide actual user value.

---

10. CORE PRODUCT MENTAL MODEL

The student should feel:

> "I am using an Industrial Engineering optimization environment to understand a problem, build a model, test assumptions, execute an optimization, and interpret the tradeoffs."

The interface should make it easy to answer:

1. What problem am I solving?
2. What model am I using?
3. What data am I using?
4. What parameters can I change?
5. What constraints are active?
6. Is the model valid?
7. What is the solver doing?
8. What result did it produce?
9. Why did the result happen?
10. What changes if I modify an assumption?
11. How does this compare with another scenario?

---

11. HCD PRINCIPLES

Autonomy

Students should control:

- parameters
- assumptions
- constraints
- scenarios
- execution
- experimentation
- comparison

Avoid unnecessary forced progression.

Competence

The system should help students understand:

- what they are doing
- whether the model is valid
- what is wrong
- what the solver is doing
- what the result means
- what changed

Cognitive Load Management

Use:

- progressive disclosure
- logical grouping
- clear hierarchy
- consistent terminology
- sensible defaults
- persistent context
- inline validation

Explainability

Whenever the system can support it, communicate:

```
Input
→ Constraint
→ Model
→ Solver
→ Result
→ Engineering Meaning
```

Professionalism

The product should feel:

- precise
- analytical
- calm
- modern
- credible

- professional

---

12. DESIGN SYSTEM AUDIT

Before implementing the redesign, inspect the existing frontend system.

Audit:

- typography
- colors
- spacing
- radius
- shadows
- buttons
- inputs
- selects
- tables
- cards
- tabs
- accordions
- alerts
- toasts
- dialogs
- tooltips
- navigation
- maps
- charts
- loading indicators
- validation
- errors
- success states

For every major pattern classify:

- REUSE
- MODIFY
- REMOVE
- NEW

Do not blindly preserve the existing design.

Do not create a new design system solely for visual novelty.

Preserve strong existing patterns where appropriate and replace weak or gamified patterns where necessary.

---

13. INFORMATION ARCHITECTURE

Audit the existing navigation and information architecture.

Identify:

- duplicated destinations
- unclear hierarchy
- unnecessary navigation
- hidden functionality
- gamification-driven navigation
- unclear terminology
- excessive context switching
- dead ends

Then implement a simpler, more coherent information architecture.

The IA should reflect actual user goals rather than gamified progression.

The final implementation must make the new IA tangible through the real navigation and routes.

---

14. COMPLETE USER JOURNEY

The redesigned application should support a coherent flow similar to:

```
Entry
↓
Orientation
↓
Model Selection
↓
Scenario Creation / Selection
↓
Understand Model
↓
Configure
↓
Validate
↓
Execute
↓
Analyze
```

```
↓
Experiment
↓
Compare
↓
Save / Revisit
```

Adapt this to the actual product.

Support alternate and recovery paths where appropriate.

---

15. EDUCATIONAL EXPERIENCE

Teach through the interaction itself.

Use:

- contextual explanations
- engineering terminology
- parameter descriptions
- model assumptions
- constraint explanations
- solver explanations
- result interpretation
- sensitivity analysis
- scenario comparison
- "what changed?" feedback

Prefer contextual information over large instructional blocks.

The interface should help the student understand **why** the system behaves as it does.

---

16. INTERACTION ARCHITECTURE

THIS IS THE CORE IMPLEMENTATION REQUIREMENT

Translate the interaction concepts normally defined for a prototype directly into production application behavior.

Every important interaction should be defined as:

```
COMPONENT
↓
STATE / VARIANT
↓
APPLICATION STATE
↓
CONDITION
↓
TRIGGER
↓
ACTION
↓
TRANSITION
↓
NEW STATE
```

For example:

```
Run Button
↓
Disabled variant
↓
isModelValid = false
↓
User clicks Run
↓
Validation condition fails
↓
Display validation feedback
↓
Remain in configuration state
```

or:

```
Run Button
↓
Ready variant
↓
isModelValid = true
↓
User clicks Run
↓
Set runStatus = "running"
↓
```

```
Show execution state
↓
Solver completes
↓
Set runStatus = "success"
↓
Display results
```

The implementation should make these relationships explicit.

---

17. APPLICATION STATE / VARIABLE ARCHITECTURE CORE IMPLEMENTATION REQUIREMENT

The application must have a clear state model.

Use the framework's native state mechanisms and existing application architecture where appropriate.

Create explicit state variables for meaningful UI/application conditions.

Examples:

```
isModelValid
isRunning
hasSolverError
hasUnsavedChanges
showExplanation
comparisonMode
isConstraintConflict

selectedModel
selectedScenario
selectedNode
runStatus
activePanel
validationMessage

constraintCount
selectedFacilityCount
objectiveValue
totalDemand
totalCapacity
```

Do not create state variables unnecessarily.

Each state variable must correspond to a meaningful application condition.

Keep state ownership clear.

Avoid unnecessary global state.

---

18. COMPONENT STATE SYSTEM

Create reusable components with meaningful states.

Example:

Model Node

- default
- hover
- focus
- selected
- valid
- warning
- error
- disabled
- processing

Parameter Input

- empty
- focused
- filled
- invalid
- warning
- disabled
- read-only

Execute Action

- ready
- disabled
- validation required
- running
- success
- error

Do not create variants merely for cosmetic differences.

Every state must correspond to real application behavior.

---

19. INTERACTION SPECIFICATION

For every primary interaction define:

Trigger

Examples:

- click
- drag
- hover
- keyboard
- form submission
- selection
- change
- scroll
- navigation
- completion of async operation

Action

Examples:

- change state
- update parameter
- update model
- validate
- show/hide UI
- navigate
- open panel/dialog
- execute model
- update result
- save scenario
- compare scenarios

Condition

Define the exact state condition.

Destination

Define the resulting application state or route.

Transition

Specify:

- interaction behavior
- timing where appropriate
- animation where useful
- immediate vs delayed feedback

---

20. INTERACTION MATRIX

Maintain an implementation-level interaction matrix during development.

Example:

ID	Current State	Trigger	Condition	Action	Result
INT-01	Configuration	Click Run	isModelValid=true	Set runStatus=running	Solver state
INT-02	Configuration	Click Run	isModelValid=false	Show validation	Remain in configuration
INT-03	Running	Solver success	—	Set runStatus=success	Results
INT-04	Results	Change parameter	—	Mark stale	Reconfiguration state

The matrix should remain synchronized with the implementation.

---

21. STATE MACHINE

Define explicit states for each major workflow.

Example:

```
IDLE
↓
CONFIGURING
↓
INCOMPLETE
↓
```

```
INVALID
↓
VALID
↓
VALIDATING
↓
READY
↓
RUNNING
↓
SUCCESS
↓
ANALYZING
↓
EDITING_AFTER_RUN
↓
RE-RUNNING
```

Also define:

- failure
- recovery
- cancellation if supported
- unsaved changes
- stale results
- revisit behavior

Implement these states in the actual application.

Do not document states that do not exist in the implementation.

---

22. PROTOTYPE 1 IMPLEMENTATION

OPERATIONS PROCESS MAP

Build the first complete product experience around a spatial/node-based interaction model.

Hypothesis

Spatial representation improves understanding of:

- model structure
- dependencies

- entities
- relationships
- constraints
- data flow
- causal relationships

Required experience

Demonstrate:

1. Entry
2. Home/workspace
3. Model selection
4. Scenario creation
5. Empty workspace
6. Add model entities
7. Select entity
8. Configure entity
9. Connect entities
10. Configure constraints
11. Invalid state
12. Constraint conflict
13. Valid state
14. Preflight validation
15. Execute
16. Running
17. Success
18. Error
19. Results
20. Modify
21. Re-run
22. Compare
23. Save
24. Revisit

Use actual application components/data where available.

Do not build an isolated demo page disconnected from the rest of the application.

---

23. PROTOTYPE 2 IMPLEMENTATION

ENGINEERING CONTROL CONSOLE

Build the second complete product experience around a structured analytical control interface.

Hypothesis

Structured parameter controls improve:

- numerical precision
- rapid experimentation
- constraint management
- sensitivity analysis
- comparison
- persistent context

Suggested architecture:

LEFT

- model parameters
- assumptions
- constraints
- validation
- scenario controls
- execution

RIGHT

- map/model visualization
- charts
- results
- solver state
- comparison
- explanation

Required experience:

1. Entry
2. Home/workspace
3. Model selection
4. Scenario selection/creation
5. Initial configuration
6. Parameter editing
7. Boundary value
8. Invalid input
9. Constraint conflict
10. Valid state
11. Validation
12. Execution
13. Error
14. Success
15. Results

16. Modify parameter
17. Re-run
18. Comparison
19. Save
20. Revisit

---

24. IMPORTANT: IMPLEMENT BOTH DIRECTIONS AS REAL APPLICATION EXPERIENCES

The two prototypes are not static concepts.

Implement them as functional experiences in the application.

They may be exposed through:

- separate routes
- a design-study switcher
- separate product modes
- clearly labeled prototype experiences

Choose the approach that best fits the existing architecture.

Do not compromise the underlying product unnecessarily.

The reviewer must be able to experience both directions directly in Replit.

---

25. RESPONSIVE / VIEWPORT

Primary target:

1440 × 900

Optimize the experience for laptop/desktop use.

At 1440 × 900:

- primary actions remain visible
- configuration context remains clear
- important outputs remain visible
- execution state is discoverable
- unnecessary horizontal scrolling is avoided

Do not over-optimize for mobile unless the existing application requires it.

---

26. MOTION AND TRANSITIONS

Use motion to communicate:

- spatial continuity
- causality
- state change
- data transformation
- system response

Do not use motion for:

- rewards
- celebrations
- gamification
- artificial excitement

Use transitions consistently.

Do not add animations simply because animation is possible.

Prefer fast, purposeful feedback for common engineering interactions.

---

27. ERROR / EMPTY / LOADING / SUCCESS STATES

Every major feature must have intentional states for:

Empty

What should the user do?

Loading

What is the system doing?

Invalid

What is wrong?

Error

What failed?

Recovery

What can the user do next?

Success

What changed?

Success must communicate system state, not reward.

---

28. ACCESSIBILITY

Implement:

- keyboard navigation
- visible focus states
- semantic labels
- sufficient contrast
- meaningful error messages
- non-color-only status communication
- readable numerical values
- accessible controls
- sensible disabled states
- reduced-motion considerations

Do not rely on color alone to communicate:

- success
- warning
- error
- validity
- selection

---

29. PROFESSIONAL VISUAL LANGUAGE

The application should feel:

- analytical
- calm

- precise
- modern
- disciplined
- credible
- technically sophisticated

Avoid:

- game aesthetics
- neon-heavy visuals
- childish illustrations
- reward animations
- unnecessary gradients
- excessive glassmorphism
- decorative dashboards
- artificial "mission control" styling
- visual noise

Make the product sophisticated through:

- information hierarchy
- typography
- spacing
- interaction quality
- data visualization
- clarity

---

30. CODEBASE IMPLEMENTATION RULES

Before writing major new code:

1. inspect existing components
2. identify reusable patterns
3. reuse existing utilities where appropriate
4. avoid unnecessary dependencies
5. preserve existing architecture where practical

Prefer:

- existing component primitives
- existing state-management patterns
- existing API clients
- existing design tokens
- existing visualization libraries

Only introduce new abstractions when justified.

Keep the implementation maintainable.

---

31. REMOVE DEAD GAMIFICATION CODE

After implementing the redesign:

- identify unused gamification routes
- identify unused gamification components
- identify obsolete state
- identify obsolete navigation
- identify obsolete imports
- identify obsolete dependencies where safe

Remove them when they are no longer required.

Do not leave broken references.

Do not perform destructive cleanup without verifying dependencies.

---

32. TEST THE ACTUAL APPLICATION

Do not consider the work complete because the code compiles.

Run the application in Replit.

Validate:

- startup
- navigation
- major workflows
- model configuration
- validation
- model execution
- results
- scenario persistence
- comparison
- redesigned interactions
- error handling
- loading states
- empty states
- gamification removal

Use browser/dev tools or available Replit testing mechanisms to inspect the real rendered experience.

33. USER JOURNEY TESTING

Test at least these scenarios:

Scenario 1 — New User

A first-time student enters the application and must understand what to do.

Scenario 2 — Configure Model

Student creates/selects a model and configures required inputs.

Scenario 3 — Invalid Model

Student creates an invalid constraint/input and must understand how to fix it.

Scenario 4 — Execute

Student successfully runs the model.

Scenario 5 — Analyze

Student interprets the results.

Scenario 6 — Experiment

Student changes a parameter and re-runs.

Scenario 7 — Compare

Student compares two scenarios.

Scenario 8 — Revisit

Student returns to previous work.

For each, verify that the application behaves coherently.

---

34. VISUAL QA

Inspect the actual rendered application at the target viewport.

Check:

- alignment
- spacing
- typography
- information hierarchy
- component consistency
- interaction affordances
- error visibility
- loading visibility
- result readability
- empty states
- navigation consistency
- absence of gamification

Fix issues found during QA.

---

35. ACCEPTANCE CRITERIA

The work is complete only when a first-time undergraduate IE student can:

1. Understand what the application does.
2. Identify the available model(s).
3. Create or select a scenario.
4. Understand the model structure.
5. Identify required inputs.
6. Configure parameters.
7. Configure constraints.
8. Detect invalid states.
9. Understand why an error occurs.
10. Correct the error.
11. Validate the model.
12. Execute the optimization.
13. Understand the execution state.
14. Interpret the result.
15. Modify assumptions.
16. Re-run the model.
17. Compare scenarios.
18. Understand what changed.
19. Save meaningful work.
20. Return to previous work.

And do so without:

- XP
- points

- badges
- quests
- leaderboards
- streaks
- levels
- reward mechanics
- artificial progression

---

36. REPLIT DELIVERABLE

The final deliverable is the **working Replit application**.

Do not make Figma files the deliverable.

Do not provide a design-only solution.

Do not stop after generating documentation.

The Replit project should contain:

- implemented redesign
- working routes
- working components
- working interactions
- functional states
- working model workflows
- redesigned navigation
- redesigned dashboards/pages
- removed gamification
- implemented error/loading/empty/success states
- tested experience

Documentation may be included in the repository where useful, but it is supporting material—not the primary deliverable.

---

37. IMPLEMENTATION REPORT

After implementation, provide a concise report containing:

What Changed

Major product-level changes.

Gamification Removed

What was removed and what replaced it.

Interaction Architecture

Major new interaction/state patterns.

State Architecture

Important state variables and state machines.

Major New Components

New reusable components introduced.

Existing Components Reused

Important reused patterns.

Technical Changes

Any backend/API/data changes and why they were necessary.

Testing Performed

What was tested and the outcome.

Known Limitations

Anything that could not be implemented fully.

---

38. DO NOT CLAIM COMPLETION WITHOUT VERIFICATION

Before declaring the work complete:

- run the application
- inspect the major routes
- exercise the primary workflows
- verify the redesigned interaction states

- verify gamification removal
- verify no major console/runtime errors
- verify existing core functionality still works
- verify both prototype directions are accessible
- perform visual QA

If something remains incomplete, explicitly identify it.

---

39. FINAL PRIORITY ORDER

When tradeoffs occur, prioritize:

1. Technical correctness
2. Working application
3. User understanding
4. Educational value
5. Cognitive-load reduction
6. User autonomy
7. Explainability
8. Interaction quality
9. Accessibility
10. Visual polish

Do not sacrifice product functionality for visual sophistication.

---

40. FINAL DIRECTIVE

Do not approach this assignment as:

> "Design a new UI."

Approach it as:

> **"Re-architect and implement the complete user experience of Network Optimization Studio directly in Replit."**

The repository audit tells you what currently exists.

HCD tells you what should improve.

The two prototype directions give you two alternative interaction architectures.

But the final output is not a document.

It is the **working product**.

Every important interaction should be implemented as:

```
COMPONENT
↓
STATE
↓
APPLICATION VARIABLE
↓
CONDITION
↓
TRIGGER
↓
ACTION
↓
TRANSITION
↓
NEW STATE
```

Both interaction architectures must be usable directly in the Replit application.

Build it.

Run it.

Test it.

Fix it.

Then deliver the working application.